

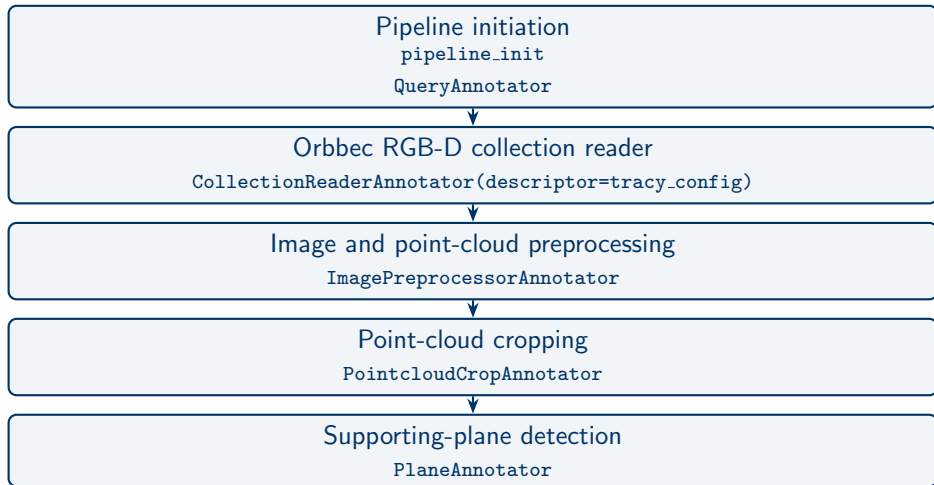
Acrambly: Perception

Mid-term Presentation

Oct 2 2026

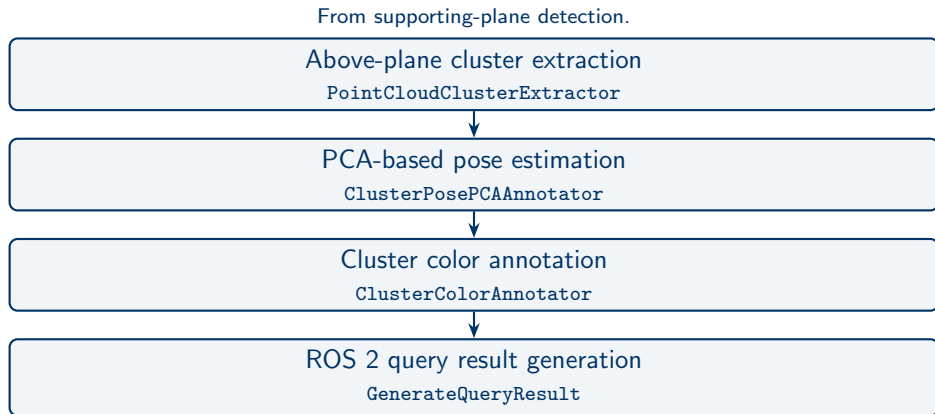
- 1 Perception pipeline 1: White
- 2 Perception pipeline 2: Blue
- 3 Storage Red
- 4 CRAM and RoboKudo integration

Perception Pipeline 1: White (1/2)



Continues with above-plane cluster extraction.

Perception Pipeline 1: White (2/2)

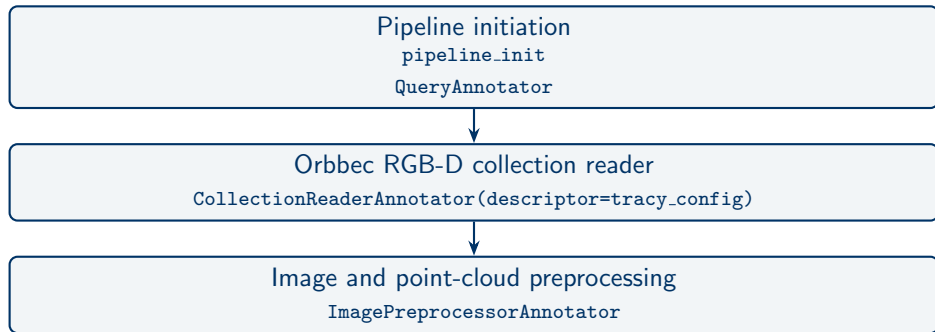


Observed behavior

The pipeline demonstrated the complete path from RGB-D acquisition to object hypotheses with pose and color information.

The pipeline was falsely detecting large box of grey and white (for reflective areas) of the table. While missing requested colored cubes.

Preception Pipeline 2: Blue(1/2)



Continues with query-selected HSV segmentation.

Preception Pipeline 2: Blue(2/2)

From image and point-cloud preprocessing.

Query-selected HSV segmentation
Largest matching image contour
ImageClusterExtractor



Corresponding 3D points from depth data
Bounding-box pose and size estimation
ClusterPoseBBAnnotator



ROS 2 query result generation
GenerateQueryResult

Observed behavior

In the performed demonstrations, this pipeline reliably detected the queried colored cube and returned its bounding-box pose.

It precisely detected even the pose of smaller cubes.

It provided the perception output required by the subsequent CRAM/Coraplex integration.

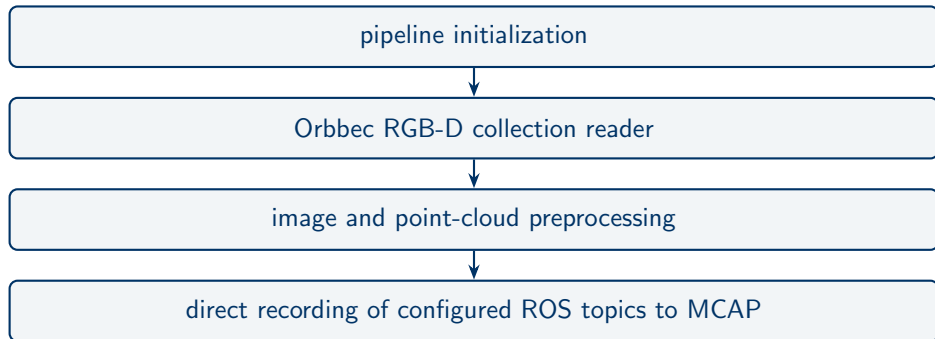
Storage Red: why MCAP?

- **Motivation:** On the newer Linux kernel used for this project, the original RoboKudo MongoDB recording workflow encountered a compatibility problem.
- **Response:** `storage_red.py` is a dedicated RoboKudo analysis engine with a `McapRecorderAnnotator`. It records the sensor input needed to reproduce a perception run in an MCAP bag.

Storage Red: previous and current recording

Aspect	Previous: <code>storage.py</code>	New: <code>storage_red.py</code>
Storage backend	MongoDB through <code>StorageWriter</code>	ROS 2 <code>rosbag2</code> with MCAP through <code>McapRecorderAnnotator</code>
Data saved	Processed CAS data, views, and world model state	Original RGB-D, camera-info, and transform ROS messages
Recording behavior	Writes the current CAS when the pipeline ticks	Records subscribed ROS topics continuously while the pipeline runs
Replay input	MongoDB records read back into a CAS	MCAP bag replayed as sensor topics
Advantage in the notes	More compressed	Easier implementation

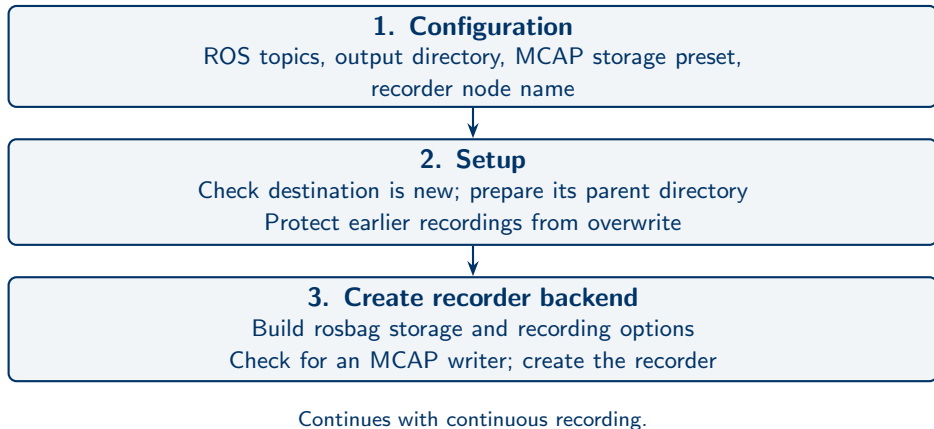
Storage Red: processing flow



Storage Red: recorded topics

Topic source	Recorded information	Purpose during replay
Orbbec depth topic	Depth image	Reconstruct three-dimensional scene data
Orbbec color topic	RGB image	Run color-based image processing
Orbbec camera-info topic	Camera calibration	Interpret and project image measurements
/tf_static	Static transforms	Recover fixed sensor-frame relationships

MCAP recorder: lifecycle (1/2)



MCAP recorder: lifecycle (2/2)

From recorder backend creation.

4. Continuous recording

Start topic discovery and message processing before the writer
Capture ROS messages continuously, not only on behavior-tree ticks



5. Status updates

Each RoboKudo tick: confirm active and report destination
Return `Status.SUCCESS`



6. Shutdown and finalization

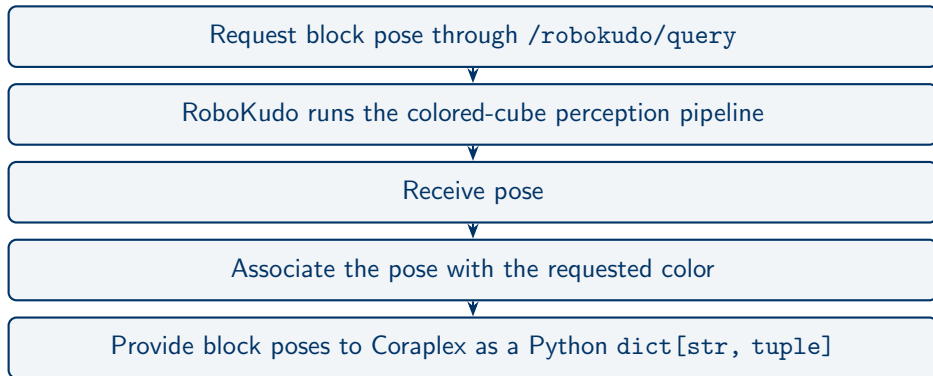
Stop recording before topic discovery and spinning
Release the backend

MCAP recorder: observed behavior

- The analysis engine recorded the configured sensor stream to an MCAP bag.
- The data was replayed successfully.
- The team also stored the recordings on Nextcloud for later experiments.

Laptop demonstration: replay the MCAP bag.

CRAM and RoboKudo integration V1



Block-stacking integration without SemDtSync.

CRAM and RoboKudo: query behavior

- **White pipeline:** Estimates all block poses at the same time in one perception pipeline run.
- **Blue pipeline:** Each ROS action goal requests object type block and one requested color. Found colors are retained; missing colors are queried again using a fresh perception result, up to the attempt limit.
- **Action-future handling:** The query works with a standalone ROS node and with a node already managed by an executor.